

Modularity and Stochastic Block Modelling

ENRICO CAPRIOGLIO

University of Sussex

Network Science Lecture 6, Spring 2026



Some Key Objectives

1. why null models are used (last week lecture's recap)
2. what is modularity and how to define it
3. learn about the Stochastic Block Model
4. compare community detection approaches
5. brief introduction to Network Inference

Why do we use network Null
Models?

Why do we use network Null
Models?

“Null models are a flexible tool to statistically benchmark the presence or magnitude of features of interest, by selectively preserving specific architectural properties of (brain) networks while systematically randomizing others.” [1]

Why do we use network Null
Models?

This is at the core of network science:

- ▶ we represent a complex system as a network
- ▶ we hypothesize that a specific network property plays a role in the *function* of the system
- ▶ we use network null models to test if the property of interest is non-trivial!

Conceptual Summary (*i.e.*, no maths)

So far, we have seen:

► **Random networks models** (Gilbert and Erdős–Rényi models)

Random networks: summary

Links are placed at random, independently of each other

- ✓ Distances between pairs of nodes are short (small-world property)
- ✗ The average clustering coefficient is much lower than on real networks of the same size and average degree
- ✗ The nodes have approximately the same degree, there are no hubs

Conclusion: the random network is not a good model of many real-world networks

Figure: Luc's summary

Conceptual Summary (*i.e.*, no maths)

So far, we have seen:

- ▶ **Random networks models** (Gilbert and Erdős–Rényi models)
- ▶ **Small-world models** (Watts-Strogatz model)

The Watts-Strogatz model [3]: summary

A regular lattice whose links are randomly rewired, with some probability p

✓ There is a range of values of the rewiring probability p for which distances between pairs of nodes are short (small-world property) and the average clustering coefficient is high

✗ The nodes have approximately the same degree, there are no hubs

Figure: Luc's summary

Conceptual Summary (*i.e.*, no maths)

So far, we have seen:

- ▶ **Random networks models** (Gilbert and Erdős–Rényi models)
- ▶ **Small-world models** (Watts-Strogatz model)
- ▶ **Preferential attachment models** (Polya's urn and variations of Barabási-Albert models)

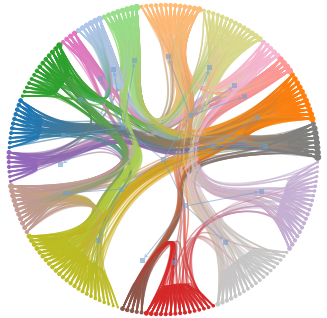
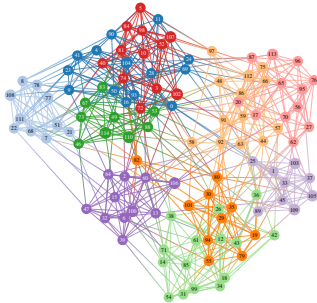
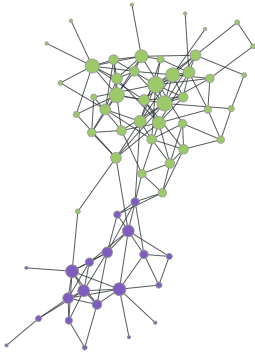
Conceptual Summary (*i.e.*, no maths)

So far, we have seen:

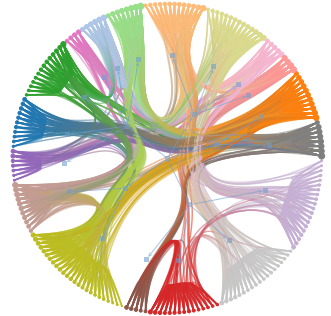
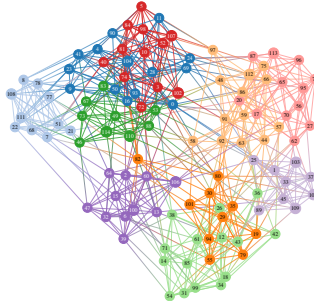
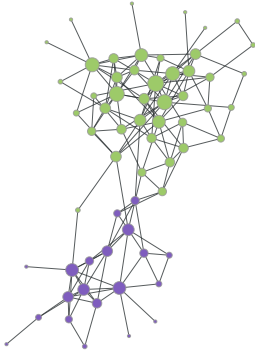
- ▶ **Random networks models** (Gilbert and Erdős–Rényi models)
- ▶ **Small-world models** (Watts-Strogatz model)
- ▶ **Preferential attachment models** (Polya's urn and variations of Barabási-Albert models)
- ▶ **Configuration model**
- ▶ **Exponential random graphs (ERG)**

What is another simple example of “whole-level” network property?

What is another simple example of “whole-level” network property?



What is another simple example of “whole-level” network property? **Modularity**



Modularity and Community Detection

Modularity seems like an intuitive concept, but it is not so easy to define.

One possible “general” definition for modules (or clusters or communities) is:

Modules

groups of vertices that have higher probability of being connected to each other than to other vertices [2]

and a network is said to have a modular structure if it has these modules.

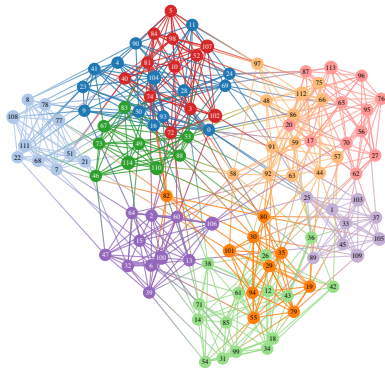


Figure: Network of American football teams [3]

Most real world networks display a modular structure:

► Sociology → Homophily Principle [4, 5]

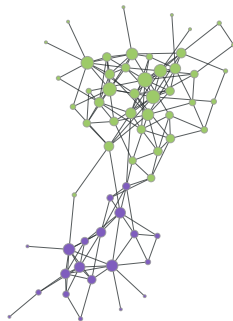


Figure: Dolphin's social network [6]

Examples

Most real world networks display a modular structure:

- ▶ Sociology → Homophily Principle [4, 5]
- ▶ Cybernetics → “The architecture of Complexity ” [7]

“Hierarchy, I shall argue, is one of the central structural schemes that the architect of complexity uses. ...” [7]

Most real world networks display a modular structure:

- ▶ Sociology → Homophily Principle [4, 5]
- ▶ Cybernetics → “The architecture of Complexity ” [7]
- ▶ Neuroscience [8], biology and so on



Figure: Hierarchically Modular Structure of the c-elegans brain

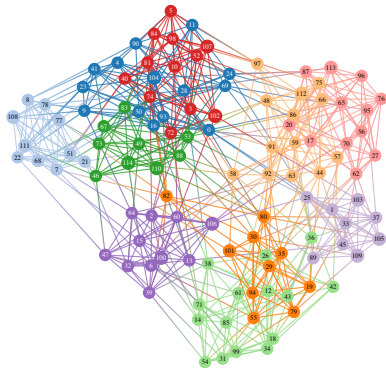
How to identify modules:

1. **Optimization based algorithms**
2. **Statistical (Bayesian) Inference**

How to identify modules:

1. **Optimization based algorithms**
2. **Statistical (Bayesian) Inference**
3. *Spectral Methods*
4. *Dynamical Methods*
5. *And many more* [9]

Why do we care?



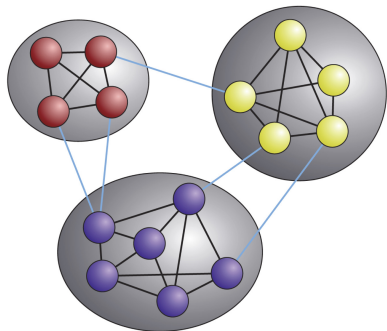


Figure: **Hard clustering:** nodes **not allowed** to be in more than 1 community

Hard vs Soft Clustering

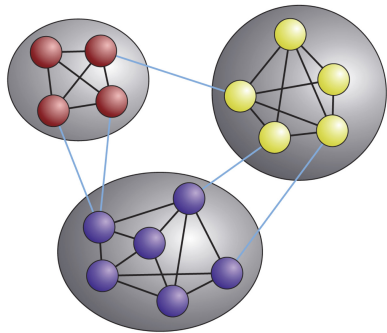


Figure: Hard clustering: nodes **not allowed** to be in more than 1 community

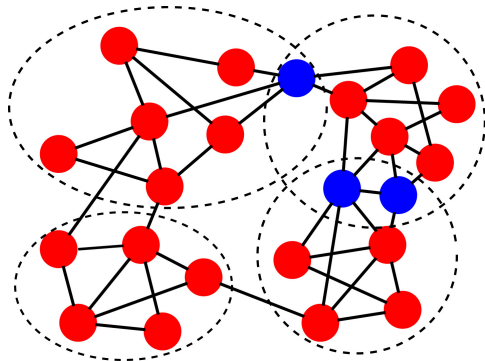


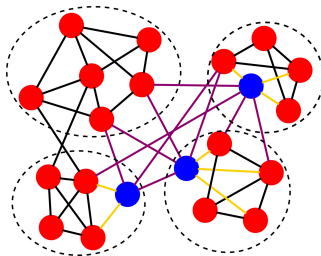
Figure: Soft clustering: nodes allowed to be in more than 1 community

Strong and Weak Communities [9] (and Ref. within)

- ▶ **Strong community:** a subgraph G_r such that the internal degree k^{in} of each vertex i is greater than its external degree k^{out} $k_i^{\text{in}}(G_r) > k_i^{\text{out}}(G_r)$
- ▶ **Weak community:** the internal degree of the subgraph exceeds its external degree $\sum_{i \in G_r} k_i^{\text{in}}(G_r) > \sum_{i \in G_r} k_i^{\text{out}}(G_r)$

Strong and Weak Communities [9] (and Ref. within)

- ▶ **Strong community:** a subgraph G_r such that the internal degree k^{in} of each vertex i is greater than its external degree k^{out} $k_i^{\text{in}}(G_r) > k_i^{\text{out}}(G_r)$
- ▶ **Weak community:** the internal degree of the subgraph exceeds its external degree $\sum_{i \in G_r} k_i^{\text{in}}(G_r) > \sum_{i \in G_r} k_i^{\text{out}}(G_r)$



Strong and Weak Communities [9] (and Ref. within)

Modern Definitions:

- ▶ **Strong community:** a subgraph each of whose vertices has a higher probability to be linked to every vertex of the subgraph than to any other vertex of the graph
- ▶ **Weak community:** a subgraph such that the average edge probability of each vertex with the other members of the group exceeds the average edge probability of the vertex with the vertices of any other group

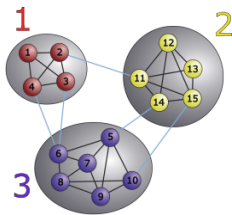
Assuming we have a good definition for the probability of an edge

Modularity Index (let's do some maths)

Idea: we want a measure (between 0 and 1) that tells us whether there are more edges between nodes in the same module than we would expect by random chance.

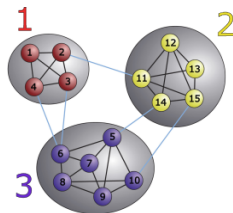
To do this:

- ▶ define what a partition is
- ▶ come up with a probability measure for what you would have expected by random chance
- ▶ compute the difference between what you have observed and what you would have expected



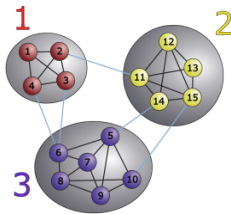
Modularity Index (let's do some maths)

- Assume we have found a **partition** of the network
 $\mathbf{b} = \{1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2\}$



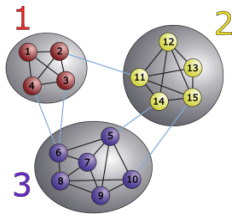
Modularity Index (let's do some maths)

- Assume we have found a **partition** of the network
 $\mathbf{b} = \{1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2\}$
- Recall the **configuration model**, our degree sequence is
 $\{k_i\} = \{3, 4, 4, 4, 5, 5, 4, 4, 4, 4, 5, 4, 3, 4, 5\}$.



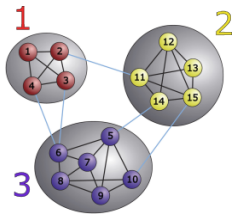
Modularity Index (let's do some maths)

- ▶ Assume we have found a **partition** of the network
 $\mathbf{b} = \{1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2\}$
- ▶ Recall the **configuration model**, our degree sequence is
 $\{k_i\} = \{3, 4, 4, 4, 5, 5, 4, 4, 4, 4, 5, 4, 3, 4, 5\}$.
- ▶ the number of edges between nodes in the same group is
 $\frac{1}{2} \sum_{i,j} A_{ij} \delta_{b_i, b_j}$



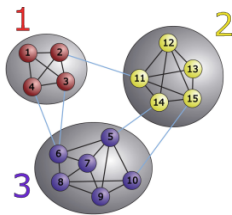
Modularity Index (let's do some maths)

- Assume we have found a **partition** of the network
 $\mathbf{b} = \{1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2\}$
- Recall the **configuration model**, our degree sequence is
 $\{k_i\} = \{3, 4, 4, 4, 5, 5, 4, 4, 4, 4, 5, 4, 3, 4, 5\}$.
- the number of edges between nodes in the same group is
 $\frac{1}{2} \sum_{i,j} A_{ij} \delta_{b_i, b_j}$
- the total expected number of edges between nodes i and j in the same group is
 $\frac{1}{2} \sum_{i,j} \frac{k_i k_j}{2L} \delta_{b_i, b_j}$, where L is the total number of edges (note the double 1/2).



Modularity Index (let's do some maths)

- Assume we have found a **partition** of the network
 $\mathbf{b} = \{1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2\}$
- Recall the **configuration model**, our degree sequence is
 $\{k_i\} = \{3, 4, 4, 4, 5, 5, 4, 4, 4, 4, 5, 4, 3, 4, 5\}$.
- the number of edges between nodes in the same group is
 $\frac{1}{2} \sum_{i,j} A_{ij} \delta_{b_i, b_j}$
- the total expected number of edges between nodes i and j in the same group is
 $\frac{1}{2} \sum_{ij} \frac{k_i k_j}{2L} \delta_{b_i, b_j}$, where L is the total number of edges (note the double $1/2$).



Modularity Index

$$Q = \frac{1}{2L} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2L} \right] \delta_{b_i, b_j}$$

For the network above $Q \approx 0.5$
 (rule of thumb: $Q > 0.3$ is considered modular)

Modularity Index (Python Example)

```
def _my_modularity(A, b):

    N = np.shape(A)[0]
    L = np.sum(A) / 2
    k_seq = _my_degree_sequence(A)

    Q = 0
    for i in range(N):
        for j in range(N):
            if b[i] == b[j]: # \delta_{b_{i}}\delta_{b_{j}} = 1
                if b[i] == b[j]
                    Q += A[i,j] - k_seq[i]*k_seq[j]/(2*L)

    return np.sum(Q) / (2*L)
```

Modularity Index (Python Example)

or more simply, using networkx

```
from networkx.algorithms.community.quality import modularity

communities = [np.where(b == i)[0] for i in range(len(np.unique(
    b)))]

modularity(g, communities)
```

NOTE: networkx uses a list of lists, in which each list specifies the nodes (labels) within a module, rather than b . Above, I converted these two equivalent descriptions.

Community Detection: Modularity Maximisation

Goal: find the partition b that maximises the modularity index Q (Similar concept to [k-means clustering](#).)

Brute force approach example:

- ▶ loop through each possible partition of a network
- ▶ compute Q for each partition
- ▶ select the partition with the highest Q

easy right?

Community Detection: Modularity Maximisation

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q (Similar concept to [k-means clustering](#).)

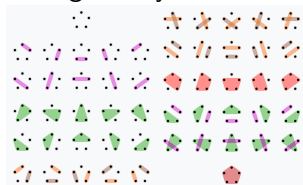
Brute force approach example:

- ▶ loop through each possible partition of a network
- ▶ compute Q for each partition
- ▶ select the partition with the highest Q

easy right?

This approach is indeed crazy:

the number of partitions of a network of size N is given by the [Bell number](#) a_n .



For instance, $a(20) = 51724158235372 \dots$

Community Detection: Modularity Maximisation

There are of course better ways to do this (greedy optimization):

- ▶ Clauset, Newman and Moore greedy optimization algorithm: [10]

```
from networkx.algorithms.community import
    greedy_modularity_communities
```

- ▶ Louvain Algorithm [11] (similar but faster):

```
from networkx.algorithms.community import
    louvain_communities
```

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

Louvain Algorithm

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

Phase 1:

Louvain Algorithm

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

Phase 1:

- ▶ Start with N communities

Louvain Algorithm

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

Phase 1:

- ▶ Start with N communities
- ▶ Sweep through the nodes, and make a proposal: assign to node i the community of one of its neighbours

Louvain Algorithm

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

Phase 1:

- ▶ Start with N communities
- ▶ Sweep through the nodes, and make a proposal: assign to node i the community of one of its neighbours
- ▶ compute Q before and after the proposed move, the difference is ΔQ

Louvain Algorithm

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

Phase 1:

- ▶ Start with N communities
- ▶ Sweep through the nodes, and make a proposal: assign to node i the community of one of its neighbours
- ▶ compute Q before and after the proposed move, the difference is ΔQ
- ▶ if ΔQ is positive, accept

Louvain Algorithm

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

Phase 1:

- ▶ Start with N communities
- ▶ Sweep through the nodes, and make a proposal: assign to node i the community of one of its neighbours
- ▶ compute Q before and after the proposed move, the difference is ΔQ
- ▶ if ΔQ is positive, accept
- ▶ repeat till no more increases

Louvain Algorithm

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

Phase 1:

- ▶ Start with N communities
- ▶ Sweep through the nodes, and make a proposal: assign to node i the community of one of its neighbours
- ▶ compute Q before and after the proposed move, the difference is ΔQ
- ▶ if ΔQ is positive, accept
- ▶ repeat till no more increases

Phase 2:

Louvain Algorithm

Goal: find the partition $\mathbf{b}$ that maximises the modularity index Q

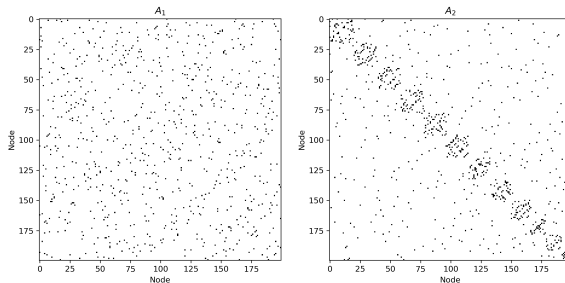
Phase 1:

- ▶ Start with N communities
- ▶ Sweep through the nodes, and make a proposal: assign to node i the community of one of its neighbours
- ▶ compute Q before and after the proposed move, the difference is ΔQ
- ▶ if ΔQ is positive, accept
- ▶ repeat till no more increases

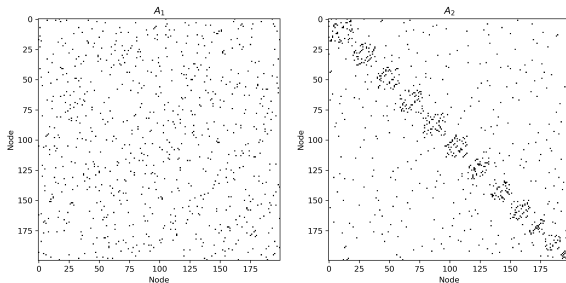
Phase 2:

- ▶ coarse-grain the system
- ▶ repeat phase 1 on the coarse-grained system

Community Detection: Modularity Maximisation Pitfall

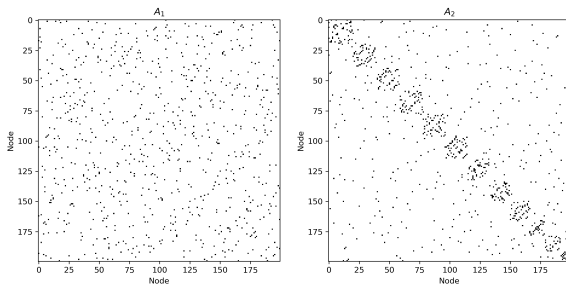


Community Detection: Modularity Maximisation Pitfall



Using Louvain, best partition gives $Q = 0.5003 \rightarrow$ for both of them!

Community Detection: Modularity Maximisation Pitfall



Using Louvain, best partition gives $Q = 0.5003 \rightarrow$ for both of them!

They are actually the same exact network, generated using a **configuration model** with degree sequence $k_i = 4, \forall i$ (same as a **regular random graph** with $d = 4$). The trick is to sort nodes using the partition from the Louvain algorithm! (revisit slide no. 3)

⚠ Danger

Using modularity maximization is almost always a **terrible idea**.

Modularity maximization is a substantially inferior method to the inference-based ones that are implemented in `graph-tool`, since it does not possess any kind of statistical regularization. Among many other problems, the method tends to massively overfit empirical data.

For a more detailed explanation see [“Modularity maximization considered harmful”](#), as well as [\[peixoto-descriptive-2023\]](#).

Do not use this approach in the analysis of networks without understanding the consequences. This algorithm is included only for comparison purposes. In general, the inference-based approaches based on `BlockState`, `NestedBlockState`, and `PPBlockState` should be universally preferred.

Figure: From `graph-tool`'s [ModularityState](#) documentation

New Goal:

Community detection without overfitting

Stochastic Block Model and Bayesian Inference

New Goal:

Community detection without overfitting

What we need: :

a suitable null model so we can ask: what is the likelihood that the network we observed is part of this class of models?

Stochastic Block Model and Bayesian Inference

New Goal:

Community detection without overfitting

What we need: :

a suitable null model so we can ask: what is the likelihood that the network we observed is part of this class of models?

Possible Solution:

- ▶ use the **Stochastic Block Model** (SBM).
- ▶ do **Bayesian inference**

Stochastic Block Model [12, 13]

Intuition: Recall the idea of homophily (“like attracts like”) in social networks, and think of modular networks as particular representations of a latent similarity space.

Stochastic Block Model [12, 13]

Intuition: Recall the idea of homophily (“like attracts like”) in social networks, and think of modular networks as particular representations of a latent similarity space.

SBM:

- ▶ given the partition of N nodes into B blocks:

$$\mathbf{b} = \{b_1, b_2, \dots, b_B\}$$

- ▶ we construct a generative model that generates networks with a probability

$$P(A|\mathbf{b})$$

Stochastic Block Model [12, 13]

Two ways to do this **((i) and (ii))**. Let B be the number of blocks and $\mathbf{n} = \{n_1, n_2, \dots, n_B\}$ the number of nodes in each block

Stochastic Block Model [12, 13]

Two ways to do this ((i) and (ii)). Let B be the number of blocks and $\mathbf{n} = \{n_1, n_2, \dots, n_B\}$ the number of nodes in each block

(i): Using the **expected** number of edges between block r and block s

$\{e_{rs}\}$ $B \times B$ matrix

$$e_{rs} = \sum_{i,j} A_{ij} \delta_{b_i,r} \delta_{b_j,s}$$

Convention is that if $r = s$ then e_{rs} is twice the number of edges

Stochastic Block Model [12, 13]

Two ways to do this ((i) and (ii)). Let B be the number of blocks and $\mathbf{n} = \{n_1, n_2, \dots, n_B\}$ the number of nodes in each block

(i): Using the **expected** number of edges between block r and block s

(ii): Using the probability (density) of connections between blocks

$\{e_{rs}\}$ $B \times B$ matrix

$\{p_{rs}\}$ $B \times B$ matrix

$$e_{rs} = \sum_{i,j} A_{ij} \delta_{b_i,r} \delta_{b_j,s}$$

Convention is that if $r = s$ then e_{rs} is twice the number of edges

Stochastic Block Model [12, 13]

Two ways to do this ((i) and (ii)). Let B be the number of blocks and $\mathbf{n} = \{n_1, n_2, \dots, n_B\}$ the number of nodes in each block

(i): Using the **expected** number of edges between block r and block s

$\{e_{rs}\}$ $B \times B$ matrix

$$e_{rs} = \sum_{i,j} A_{ij} \delta_{b_i,r} \delta_{b_j,s}$$

Convention is that if $r = s$ then e_{rs} is twice the number of edges

(ii): Using the probability (density) of connections between blocks

$\{p_{rs}\}$ $B \times B$ matrix

$$\text{if } r \neq s \quad p_{rs} = \frac{e_{rs}}{n_r \times n_s}$$

$$\text{if } r = s \quad p_{rs} = \frac{1}{2} \frac{e_{rs}}{\binom{n_r}{2}}$$

Stochastic Block Model: Example

```
import math
random.seed(8) # so you can replicate this exactly
# set up the stage (same convention as in previous slides)
n = np.array([5,5])
N = sum(n) # total no. of nodes
e = np.array([[16,3],[3,16]]) # double counting within block
edges
p = _p_from_e(e,n)
g = nx.stochastic_block_model(n, p)
```

Stochastic Block Model: Example

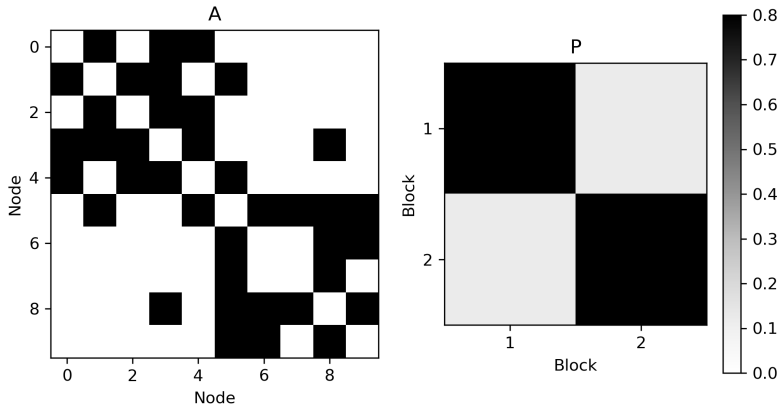
```
# just one way to do this
def _p_from_e(e,n):
    p = np.zeros(e.shape)
    for i in range(len(n)):
        for j in range(i,len(n)):

            if j == i:
                L_max = math.comb(n[i], 2) * 2
            else:
                L_max = n[i] * n[j]

            p[i,j] = e[i,j] / L_max
            p[j,i] = p[i,j]

    return p
```

Stochastic Block Model: Example



Stochastic Block Model: Example

We can of course generate much larger and interesting graphs:

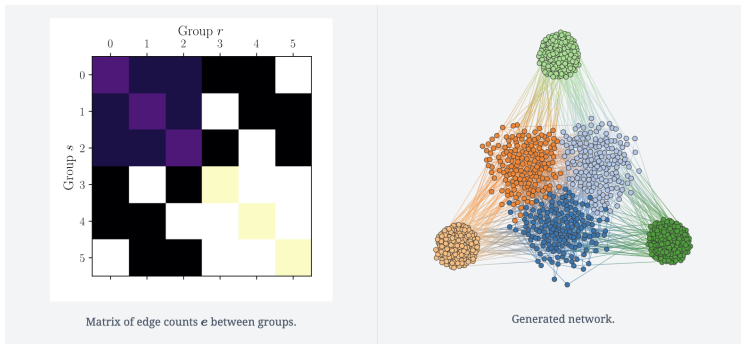


Figure: Example using graph-tool ([from graph-tool documentation](#))

We wish to learn the best partition $\mathbf{b}$ after observing a real world network A , *i.e.*, $P(\mathbf{b}|A)$.

Bayesian Inference Intuition

We wish to learn the best partition $\mathbf{b}$ after observing a real world network A , *i.e.*, $P(\mathbf{b}|A)$.

Using, $P(X \wedge Y) = P(X|Y)P(Y) = P(Y|X)P(X)$

Bayes' Rule

$$P(\mathbf{b}|A) = \frac{P(A|\mathbf{b})P(\mathbf{b})}{P(A)}$$

- ▶ $P(A|\mathbf{b})$ is the likelihood
- ▶ $P(\mathbf{b})$ is our prior
- ▶ $P(A)$ is the evidence

Bayesian Inference: SBM

Goal: infer the most likely partition $\mathbf{b}$ of the observed network A

→ maximise $p(\mathbf{b}|A) \propto P(A|\mathbf{b})P(\mathbf{b})$

Bayesian Inference: SBM

Goal: infer the most likely partition $\mathbf{b}$ of the observed network A

→ maximise $p(\mathbf{b}|A) \propto P(A|\mathbf{b})P(\mathbf{b})$

To do this we take very seriously the *principle of maximum indifference*, i.e., we want to choose the most uninformative priors and a generative model (likelihood) with maximal entropy.

Bayesian Inference: SBM

Goal: infer the most likely partition $\mathbf{b}$ of the observed network A

→ maximise $p(\mathbf{b}|A) \propto P(A|\mathbf{b})P(\mathbf{b})$

To do this we take very seriously the *principle of maximum indifference*, i.e., we want to choose the most uninformative priors and a generative model (likelihood) with maximal entropy.

Lucky for us, the SBM $P(A|\mathbf{b})$ is the maximum entropy model for a network A with partition $\mathbf{b}$ [14].

^aSee Q. 3 of Lab class 5!

Problem: We can't solve $p(\mathbf{b}|A) \propto P(A|\mathbf{b})P(\mathbf{b})$ analytically

Best solution: we can employ MCMC

Idea: Sample from $P(\mathbf{b}|A)$ by starting with some $\mathbf{b}_0$ and then making new proposals $\mathbf{b} \rightarrow \mathbf{b}'$ with probability $P(\mathbf{b}'|\mathbf{b})$. We accept the change with probability

$$\min \left(1, \frac{P(\mathbf{b}'|A)P(\mathbf{b}|\mathbf{b}')}{P(\mathbf{b}|A)P(\mathbf{b}'|\mathbf{b})} \right)$$

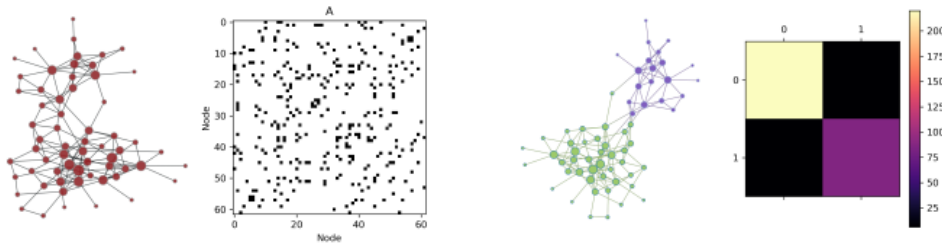
Bayesian Inference: in practice, we use graph-tool

```
import graph_tool.all as gt
# load adjacency matrix
sparse_dolphin_A = scipy.io.mmread('<your path>/soc-dolphins.mtx')
# create graph
dolphin_G = gt.Graph(sparse_dolphin_A, directed=False)
# stochastic block modelling
dolphine_state = gt.minimize_blockmodel_dl(dolphin_G) # by default degree corrected, otherwise:
                state_args=dict(deg_corr=False)
for i in range(100):
    dolphine_state.multiflip_mcmc_sweep(beta=np.inf, niter=10)
```

Where I used the dolphin's dataset [6] from [here](#).

The above is a “fancier” version of the *Metropolis-Hastings acceptance-rejection MCMC* algorithm (we used [simulated annealing](#) by setting $\beta = \infty$). To do what we described in the previous slide we use

```
gt.mcmc_sweep(beta=1.0, niter=1000)
```



My plot settings will be available on canvas!

graph-tool offers many extensions of what we studied today, in particular nested stochastic block modelling:

```
worm_G = gt.collection.data["celegansneural"]
worm_state = gt.minimize_nested_blockmodel_dl(
    worm_G)
# description length from quick likelihood
# minimization:
S0 = worm_state.entropy()
# let's decrease the description length:
for i in range(1000):
    worm_state.multiflip_mcmc_sweep(beta=np.inf,
                                    niter=10)
# description length after after sampling from
# the posterior:
S = worm_state.entropy()
worm_state.draw(ecmap = (plt.cm.inferno, .6),
               edge_pen_width = 1) # output = "c-elegans-
# network.pdf",
worm_state.print_summary()
print('\nimprovement after sampling from the
posterior:', S - S0)
```



Modularity Maximisation:

finds a partition that maximises a score

Bayesian SBM:

finds partitions that make the graph most probable under a generative model

What we have not covered

but very worth mentioning.

What we have not covered

but very worth mentioning.

Community detection:

- ▶ Dynamical Methods: random-walk, diffusion, or synchronization
- ▶ Spectral Methods (i.e., linear algebra)

What we have not covered

but very worth mentioning.

Community detection:

- ▶ Dynamical Methods: random-walk, diffusion, or synchronization
- ▶ Spectral Methods (i.e., linear algebra)

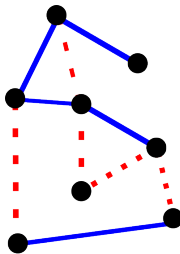
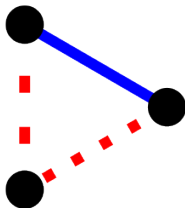
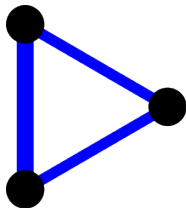
Community formation:

How do communities emerge in the first place? Polarization, segregation, echo-chambers

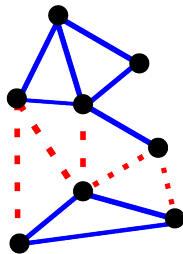
Emergence of polarization

Consider the ancient proverbs:

“the enemy of my enemy is my friend” and *“the friend of my friend is my friend”*



$t=0$



$t=4$

1. Váša F and Mišić B. Null models in network neuroscience. *Nature Reviews Neuroscience* 2022; 23 (8):493–504. DOI: [10.1038/s41583-022-00601-9](https://doi.org/10.1038/s41583-022-00601-9)
2. Fortunato S. Community detection in graphs. *Physics Reports* 2010; 486 (3–5):75–174. DOI: [10.1016/j.physrep.2009.11.002](https://doi.org/10.1016/j.physrep.2009.11.002)
3. Evans TS. Clique graphs and overlapping communities. *Journal of Statistical Mechanics: Theory and Experiment* 2010; 2010 (12):P12037. DOI: [10.1088/1742-5468/2010/12/p12037](https://doi.org/10.1088/1742-5468/2010/12/p12037)
4. Granovetter MS. The strength of weak ties. *American journal of sociology* 1973; 78 (6):1360–80

Bibliography II

5. McPherson M, Smith-Lovin L, and Cook JM. Birds of a Feather: Homophily in Social Networks. *Annual Review of Sociology* 2001; 27 (1):415–44. DOI: [10.1146/annurev.soc.27.1.415](https://doi.org/10.1146/annurev.soc.27.1.415)
6. Lusseau D. The emergent properties of a dolphin social network. *Proceedings of the Royal Society of London. Series B: Biological Sciences* 2003; 270 (suppl_2):S186–S188
7. Simon HA. The architecture of complexity. *The Roots of Logistics*. Springer, 2012 :335–61
8. Meunier D. Hierarchical modularity in human brain functional networks. *Frontiers in Neuroinformatics* 2009; 3. DOI: [10.3389/neuro.11.037.2009](https://doi.org/10.3389/neuro.11.037.2009)

Bibliography III

9. Fortunato S and Hric D. Community detection in networks: A user guide. *Physics Reports* 2016; 659:1–44. DOI: [10.1016/j.physrep.2016.09.002](https://doi.org/10.1016/j.physrep.2016.09.002)
10. Clauset A, Newman MEJ, and Moore C. Finding community structure in very large networks. *Physical Review E* 2004; 70 (6). DOI: [10.1103/physreve.70.066111](https://doi.org/10.1103/physreve.70.066111)
11. Blondel VD, Guillaume JL, Lambiotte R, and Lefebvre E. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008; 2008 (10):P10008. DOI: [10.1088/1742-5468/2008/10/p10008](https://doi.org/10.1088/1742-5468/2008/10/p10008)
12. Peixoto TP. Bayesian Stochastic Blockmodeling. 2019. DOI: [10.1002/9781119483298.ch11](https://doi.org/10.1002/9781119483298.ch11)

13. Peixoto TP. Descriptive vs. Inferential Community Detection in Networks: Pitfalls, Myths and Half-Truths. Cambridge University Press, 2023. DOI: [10.1017/9781009118897](https://doi.org/10.1017/9781009118897)
14. Peixoto TP. Entropy of stochastic blockmodel ensembles. Physical Review E—Statistical, Nonlinear, and Soft Matter Physics 2012; 85 (5):056122